

O Canivete Suíço Claude Code

Guia completo dos recursos nativos do Claude Code —
explicados de forma acessível e aterrissados na stack,
na jornada e na doutrina do Meu Cumpadre, com
exemplos reais.

"O diamante é carbono que não desistiu da pressão."
Lucro é combustível · Causa é destino · Jogo é infinito.

Sumário

- 01** Como usar este guia
Legenda, convenções e a régua D>C>V ao operar o Code

- 02** O que é o Claude Code
Anatomia, onde ele roda, o loop agente-ferramenta

- 03** CLAUDE.md — a memória do projeto
Hierarquia de memória, imports, o nosso próprio CLAUDE.md

- 04** Comandos de barra nativos
Catálogo completo dos /comandos embutidos

- 05** Gerenciamento de contexto
/compact, /clear, auto-compactação e o Context Budget MC

- 06** Skills (Agent Skills)
A família de skills do MC: /diario, /watchdog, /squad-r3...

- 07** Comandos customizados
.claude/commands, argumentos e atalhos de prompt

- 08** Subagentes
sister-ancia, contexto fresco e paralelismo

- 09** Hooks
mc-lint como Stop hook block-mode — enforcement determinístico

- 10** MCP — Model Context Protocol
github, Supabase, n8n, Adobe na nossa mesa

- 11** Plan Mode e Extended Thinking
Planejar antes de executar; pensar mais fundo

- 12** Permissões e settings.json
allow / deny / ask — o nosso Workaround Classifier

- 13** Sessões, checkpoints e rewind
--continue, --resume e desfazer com segurança

- 14** Modo headless e SDK
claude -p, automação, JSON — o Code como peça de pipeline

- 15** Claude Code na web e ambientes
A superfície onde este guia está sendo gerado

- 16** Integração com GitHub

17 IDE, atalhos e modo Vim

VS Code, JetBrains, teclado

18 Ferramentas nativas

Bash, Read, Edit, Glob, Grep, WebSearch, Task...

19 Output styles, status line e /fast

Customização de comportamento e velocidade

20 /loop e recorrência

O recurso da semana — tarefas em intervalo

21 Apêndice A — Mapa do Canivete

Recurso → aplicação direta no MC

22 Apêndice B — Régua MC ao operar o Code

D>C>V, LGPD, Firewall OAB na prática

01 Como usar este guia

Este material é um mapa de capacidades. Cada recurso nativo do Claude Code é explicado em três camadas: **o que é**, **como se usa** (comando/sintaxe exata) e **como se aplica ao Meu Cumpadre** com exemplo real. Você não precisa decorar — precisa saber que a ferramenta existe e onde ela encaixa.

Legenda dos blocos

◆ APLICAÇÃO MC

Como o recurso serve à nossa causa, jornada E0-E7 ou doutrina. Exemplos usam coisas que já existem no nosso repositório.

✓ BOA PRÁTICA

Forma recomendada de usar — alinhada às convenções MC.

▲ ATENÇÃO

Cuidado operacional, custo de contexto ou pegadinha comum.

× LINHA VERMELHA

Onde uma Proibição Absoluta ou regra inviolável do MC se aplica.

A régua antes de tudo

Toda capacidade descrita aqui opera **sob** a hierarquia inviolável do MC. Poder técnico nunca passa por cima da doutrina:

× HIERARQUIA INVIOLÁVEL

Dignidade > Compliance > Viabilidade. Nenhum recurso do Claude Code — por mais conveniente — autoriza exercer advocacia (Firewall OAB), prometer prazo de concessão INSS, colocar senha gov.br fora do Bitwarden, ou fabricar dado sem fonte (Proof-First). A ferramenta amplia o operador; não suspende a regra.

02 O que é o Claude Code

Claude Code é o agente de engenharia da Anthropic que opera **dentro do seu ambiente** — terminal, IDE, app desktop, web ou GitHub Actions. Diferente de um chat, ele **lê arquivos, edita código, roda comandos e usa ferramentas** em um loop: pensa → age (chama uma ferramenta) → observa o resultado → repete, até concluir a tarefa.

Onde ele roda (superfícies nativas)

Superfície	Como se acessa	Uso típico no MC
Terminal (CLI)	<code>claude</code> na pasta do projeto	Mesa de trabalho do Founder em TALÃO
App Desktop (Mac/Win)	Aplicativo Claude	Sessões longas de cunhagem
Web	<code>claude.ai/code</code>	Superfície onde este guia foi gerado
Extensão de IDE	VS Code / JetBrains	Backend NestJS do Igor
GitHub Actions	workflow CI	Revisão automática de PR

O modelo mental: agente, não autocomplete

O Code decide quais ferramentas chamar e em que ordem. Ele pode rodar várias ferramentas independentes ao mesmo tempo, delegar sub-tarefas a subagentes, e parar para pedir confirmação em ações arriscadas. Tudo isso é configurável — e é exatamente o que este guia destrincha.

◆ APLICAÇÃO MC

No MC o Claude Code é a instância **Code TALÃO** (Arquiteto + Executor), operada exclusivamente pelo Founder Alessandro (cabeça única #50.2). Beto **não** opera Code/git/TALÃO (Errata #10). O Code escreve no Cérebro 1 (Documentação) livremente, mas só escreve no Vault (Cérebro 2) sob gate "aprovado para vault" (ADR-011).

03 CLAUDE.md — a memória do projeto

O `CLAUDE.md` é um arquivo que o Claude Code lê automaticamente no início de cada sessão e injeta como instrução de sistema. É a memória persistente do projeto — regras, convenções, vocabulário, proibições. **O MC já vive disso:** o nosso `CLAUDE.md` carrega D>C>V, Firewall OAB, precificação corrente e as Proibições Absolutas.

Hierarquia de memória (do mais amplo ao mais específico)

Nível	Arquivo	Escopo
Usuário	<code>~/.claude/CLAUDE.md</code>	Vale para todos os seus projetos
Projeto	<code>./CLAUDE.md</code> (raiz)	Compartilhado no repo, versionado em git
Local	<code>./CLAUDE.local.md</code>	Seu, não versionado (preferências pessoais)
Subpasta	<code>./pasta/CLAUDE.md</code>	Carregado ao trabalhar naquela subárvore

Imports com @

Um `CLAUDE.md` pode puxar outros arquivos com `@caminho/arquivo.md`, mantendo o arquivo principal enxuto:

```
Veja as convenções de backend em @docs/convencoes-nestjs.md
Regras de custódia: @03-GOVERNANCA/ADR-009a.md
```

Editar a memória

Use o comando `/memory` para abrir e editar os arquivos de memória, ou comece uma frase com `#` que o Code pergunta em qual `CLAUDE.md` gravar aquilo. `/init` gera um `CLAUDE.md` inicial analisando o codebase.

◆ APLICAÇÃO MC

Nosso `CLAUDE.md v3.0` é o coração doutrinário. Ele passou pelo refactor enxuto (Fase 3) e aponta para o Vault como fonte de verdade. **Refatorar o CLAUDE.md é cunhagem fundacional → exige Rito R2** (Founder + C3.1 + Anciã). Não se mexe nele em fluxo operacional comum.

▲ ATENÇÃO

`CLAUDE.md` inteiro entra no contexto toda sessão. Mantê-lo enxuto (como já fizemos) preserva Context Budget. Detalhe vai para o Vault e entra por import/ponteiro, não inline.

04 Comandos de barra nativos

Comandos de barra (`/comando`) são ações rápidas digitadas no prompt. Os nativos controlam a sessão, o contexto, o modelo, permissões e integrações. Abaixo, o catálogo dos principais.

Sessão e contexto

Comando	O que faz
<code>/clear</code>	Zera o contexto da conversa (recomeça limpo)
<code>/compact</code>	Resume o histórico para liberar contexto sem perder o fio
<code>/resume</code>	Retoma uma sessão anterior pela lista
<code>/export</code>	Exporta a conversa atual
<code>/cost</code>	Mostra uso de tokens/custo da sessão
<code>/status</code>	Estado da sessão, conta, conexões

Configuração e ambiente

Comando	O que faz
<code>/config</code>	Abre as configurações (tema, modelo, etc.)
<code>/model</code>	Troca o modelo (Opus / Sonnet / Haiku)
<code>/fast</code>	Liga/desliga o modo rápido (Opus com saída acelerada)
<code>/permissions</code>	Gerencia regras de permissão (allow/deny/ask)
<code>/add-dir</code>	Adiciona outra pasta ao escopo de trabalho
<code>/vim</code>	Liga o modo de edição estilo Vim no prompt
<code>/doctor</code>	Diagnostica a instalação e a saúde do ambiente

Memória, extensões e integrações

Comando	O que faz
<code>/init</code>	Gera um CLAUDE.md analisando o codebase
<code>/memory</code>	Edita os arquivos de memória (CLAUDE.md)
<code>/agents</code>	Cria e gerencia subagentes
<code>/mcp</code>	Lista e gerencia servidores MCP conectados
<code>/hooks</code>	Configura hooks de ciclo de vida
<code>/review</code>	Revisa um pull request
<code>/login</code> · <code>/logout</code>	Autenticação da conta
<code>/help</code>	Ajuda e lista de comandos
<code>/bug</code> · <code>/feedback</code>	Reporta problema / envia feedback à Anthropic

Contexto, planejamento e automação

Comando	O que faz
<code>/context</code>	Mostra o que está ocupando a janela de contexto
<code>/recap</code>	Resume o trabalho até aqui sem limpar o contexto
<code>/usage</code>	Uso de tokens e rastreamento de custo
<code>/plan</code>	Entra no Plan Mode (exploração read-only)
<code>/effort</code>	Ajusta o orçamento de raciocínio (low/medium/high)
<code>/branch</code>	Bifurca a sessão atual em uma nova
<code>/batch</code>	Decompõe uma mudança grande em tarefas paralelas
<code>/tasks</code> · <code>/background</code>	Lista trabalho de subagentes / roda a sessão em segundo plano
<code>/schedule</code> · <code>/loop</code> · <code>/goal</code>	Tarefa recorrente · repetir em intervalo · rodar até cumprir condição
<code>/plugin</code>	Instala e gerencia plugins
<code>/desktop</code> · <code>/teleport</code>	Passa a sessão para o Desktop / puxa sessão web ou iOS para o terminal

◆ APLICAÇÃO MC

Note que **as nossas skills aparecem como comandos de barra**: `/diario` , `/watchdog` , `/status-caso` , `/novo-cliente` , `/squad-r3` . Quando o operador digita `/watchdog` , está acionando a skill homônima (capítulo 6).

▲ **ATENÇÃO**

A lista exata pode variar com a versão do Code. Em caso de dúvida, `/help` sempre mostra o que existe na sua instalação — não chute um comando.

05 Gerenciamento de contexto

O Claude tem uma janela de contexto finita. Conforme a conversa cresce, ela enche. Gerenciar contexto é manter o agente afiado: nem esquecido, nem afogado em ruído.

Os três mecanismos

- **Auto-compactação:** quando o contexto fica cheio, o Code resume automaticamente o histórico e continua — você não precisa parar a tarefa no meio.
- **/compact manual:** você força o resumo num ponto bom (fim de uma sub-tarefa), preservando as conclusões e descartando o ruído.
- **/clear:** recomeço total — use ao trocar completamente de assunto.

◆ APLICAÇÃO MC

O nosso CLAUDE.md define **Context Budget**: sessão em 30-40%, **/compact** ou **/clear** quando passar de 60%, e fluxo **Spec-Driven** (Research → PRD → Spec → Implement em sessões separadas). Isso evita que um caso de cidadão e uma cunhagem de ADR contaminem o mesmo contexto.

✓ BOA PRÁTICA

Dê **/compact** ao terminar uma etapa da jornada (ex.: fechou a triagem E2, vai para a coleta E3) e **/clear** ao começar um caso novo. Contexto limpo = menos alucinação, mais Proof-First.

06 Skills (Agent Skills)

Skills são pacotes de conhecimento e procedimento que o Code carrega **sob demanda**. Cada skill é uma pasta com um `SKILL.md` que tem um frontmatter (nome + descrição + frases-gatilho) e o corpo com instruções. O Code lê só a descrição até precisar — e aí carrega a skill inteira. É a forma nativa de industrializar um procedimento repetível.

Anatomia de uma skill

```
---
name: watchdog
description: Monitora todos os casos na fase E5 (aguardando análise
  INSS) e gera relatório de pendências e alertas.
  Frases-gatilho: "watchdog", "casos pendentes INSS", "fase E5".
---
# instruções detalhadas do procedimento...
```

Como se aciona

Três formas: (1) digitando `/watchdog` como comando; (2) escrevendo uma frase-gatilho ("o que está parado no INSS?"); (3) o Code reconhece o padrão da descrição e a invoca sozinho.

◆ APLICAÇÃO MC

O MC já tem uma **família de skills modulares** — cada uma é um pedaço do que antes era o monólito "orquestrador-mestre" (aposentado em 2026-06-03 para não ser segunda fonte de verdade):

- `/novo-cliente` · onboarding E1 com DIB Urgency para B31/B91/B42/B43
- `/status-caso` e `/watchdog` · acompanhamento E5
- `/squad-r2` · rito de cunhagem fundacional (ADR/Princípio)
- `/squad-r3` · parecer adversarial de compliance (OAB/LGPD)
- `/mc-pre-selagem` · gate antes de escrever no Vault
- `/mc-dossie-proof-first` · linha de evidência com fonte:linha + SHA-256
- `/diario` , `/handoff-juridico` , `/checklist-docs` ...

✓ BOA PRÁTICA

Uma boa `description` com frases-gatilho é o que faz o Code escolher a skill certa na hora certa. É engenharia de descoberta — não é decoração.

07 Comandos de barra customizados

Além das skills, você pode criar comandos de barra simples como arquivos Markdown em `.claude/commands/`. O nome do arquivo vira o comando. É o jeito leve de salvar um prompt que você repete.

Como criar

Crie `.claude/commands/revisar-contrato.md`:

```
---
description: Revisa uma minuta contra o Firewall OAB
---
Revise a minuta a seguir contra as Proibições Absolutas do MC,
em especial promessa de resultado e success fee no §1.
Minuta: $ARGUMENTS
```

Aí no prompt: `/revisar-contrato minuta v2.4`. O texto após o comando substitui `$ARGUMENTS`.

Comando vs. Skill — qual usar?

Use comando customizado	Use skill
Um prompt curto e fixo que você repete	Procedimento com várias etapas, regras e arquivos de apoio
Sem lógica de descoberta automática	Quer que o Code invoque sozinho por contexto

◆ APLICAÇÃO MC

O MC optou por **skills** para quase tudo (procedimentos doutrinários ricos, com gates e frases-gatilho). Comandos customizados servem para atalhos simples — ex.: um `/teste-zilda` que aplica o Teste Zilda-STF a um texto colado.

08 Subagentes

Um subagente é um Claude separado, com seu próprio contexto, ferramentas e (opcionalmente) modelo, que o agente principal aciona para uma sub-tarefa. Ele trabalha isolado e devolve só a conclusão — sem entupir o contexto principal com a investigação inteira.

Para que servem

- **Contexto fresco e sem viés:** um revisor que não viu você escrever o texto julga melhor.
- **Paralelismo:** várias buscas independentes ao mesmo tempo.
- **Economia de contexto:** a varredura pesada fica no subagente; volta só o resultado.

Como se define

Um arquivo em `.claude/agents/nome.md` com frontmatter. Gerencie via `/agents`.

```
---
name: sister-ancia
description: Revisor adversarial doutrinário. Aciona em contexto
  FRESCO para revisar artefatos/diffs MC contra drift doutrinário.
tools: Read, Grep, Glob, Bash
model: opus
---
# os 9 eixos de revisão adversarial...
```

◆ APLICAÇÃO MC

A **Sister Anciã** é exatamente isso: subagente real (`model: opus` , ferramentas só de leitura) que revisa artefatos **antes de selar** contra os 9 eixos — D>C>V, Firewall OAB, Proof-First, LGPD, Inversão Cósmica, valores superados, confusão de instância, disciplina de gate/versão. Ela **reporta gaps, não conserta**. Temos também o `mc-grok-context-engineer` . A skill `/mc-pre-selagem` orquestra a Anciã + o mc-lint num veredito único.

✓ BOA PRÁTICA

Subagente para revisão deve ter contexto fresco e ferramentas mínimas (só leitura). É o desenho da Anciã: ela não pode escrever, só apontar.

09 Hooks

Hooks são comandos shell que o Claude Code executa **automaticamente** em momentos do ciclo de vida — não dependem de o agente "lembrar". É a diferença entre orientar e enforçar: o hook é determinístico.

Eventos disponíveis

Evento	Quando dispara
PreToolUse	Antes de uma ferramenta rodar (pode bloquear)
PostToolUse	Depois de uma ferramenta rodar
UserPromptSubmit	Quando você envia um prompt
Stop	Quando o agente vai encerrar o turno (pode bloquear)
SessionStart	No início da sessão (ex.: preparar testes)
SubagentStop / Notification	Fim de subagente / notificações

Como se configura

No `settings.json`, sob `"hooks"`. Saída do hook com código de erro em modo bloqueante **impede** a ação e devolve a mensagem como feedback.

```
"hooks": {
  "Stop": [
    { "hooks": [
      { "type": "command",
        "command": "MC_LINT_MODE=block python3 .claude/hooks/mc-lint.py" }
    ] }
  ]
}
```

◆ APLICAÇÃO MC

Este é **literalmente o nosso settings.json**. O `mc-lint.py` roda como **Stop hook em block-mode**: ao fim de cada turno ele varre as linhas adicionadas e **bloqueia o encerramento** se encontrar uma Proibição Absoluta (R\$ 1.500 standard, Φ₂ como cobrança, "Juliana Alencar", Autentique, Router-Ethics 9.0...), listando `label · arquivo:linha · correção`. Temos também o `mc-audit.py`. É enforcement, não conselho — o gate é mecânico (gate R1, Zona Verde, varre só .md/.txt/.json, sem PII).

× LINHA VERMELHA

O hook é a última cerca. Mas ele é regex de tokens — não substitui a revisão semântica da Anciã nem o gate humano R3 para OAB/LGPD/PII. Camadas, não atalho.

10 MCP — Model Context Protocol

MCP é o padrão aberto que conecta o Claude Code a **sistemas externos** — bancos de dados, APIs, ferramentas SaaS — via "servidores" que expõem ferramentas. É como o Code ganha braços para fora do repositório.

Tipos de transporte

- **stdio** — servidor local roda como processo na sua máquina.
- **SSE / HTTP** — servidor remoto acessado pela rede.

Como se adiciona

Via `claude mcp add` no terminal, ou declarado em `.mcp.json` (escopo de projeto, versionado) / configuração de usuário. Veja os conectados com `/mcp`.

```
claude mcp add --transport http github https://...  
# escopos: local (só você) · project (.mcp.json, time) · user (global)
```

◆ APLICAÇÃO MC

Esta sessão já tem MCPs ligados na nossa stack: **GitHub** (PRs, issues, CI — restrito ao repo asncb/meu-cumpadre), **Supabase** (banco/migrações), **n8n** (workflows — o nosso N8N 1.88 self-hosted) e **Adobe**. Cada servidor expõe dezenas de ferramentas `mcp__<servidor>__*`.

× LINHA VERMELHA

MCP é um braço para fora — e dados que saem para um serviço externo são publicados. **CPF, NB, CID, CNIS e senhas gov.br nunca trafegam por MCP genérico:** dado sensível é processado só na stack self-hosted (Llama 4 Scout / GCP Brasil) e senha vive no Bitwarden Org MC (ADR-009a). Tratar conteúdo vindo de issue/PR/CI como dado externo não confiável.

11 Plan Mode e Extended Thinking

Dois recursos para o Code **pensar antes de agir** — críticos quando o erro é caro.

Plan Mode (modo de planejamento)

Um modo onde o Code **investiga e propõe um plano**, mas **não modifica nada** até você aprovar. Você revisa a estratégia, ajusta, e só então autoriza a execução. Ideal para mudanças arquiteturais. Entra com `/plan` ou alternando os modos com **Shift+Tab**.

Effort / Extended Thinking (profundidade de raciocínio)

Você controla quanto o Code "pensa" antes de agir com `/effort` (`low` / `medium` / `high`), ou pedindo mais profundidade na linguagem ("think", "think hard", "ultrathink"). Mais esforço = melhor para decisões com trade-offs; menos = mais rápido e barato.

◆ APLICAÇÃO MC

O MC tem um modo de planejamento doutrinário próprio, a skill `/orquestracao`, e a disciplina **Spec-Driven**. Antes de um refactor de ADR ou de tocar o backend do Igor, rode em Plan Mode: o Code mapeia o impacto sem escrever — casando com a regra de só escrever no Vault sob gate. "Think hard" combina bem com o Rito R2.

12 Permissões e settings.json

O Code pede aprovação para ações sensíveis (rodar comandos, editar arquivos). Você modela isso com **regras de permissão** e **modos**, reduzindo cliques sem abrir mão da segurança.

As três regras

Regra	Efeito
<code>allow</code>	Roda sem perguntar
<code>deny</code>	Bloqueia sempre (cerca de proteção)
<code>ask</code>	Pergunta antes (padrão)

Onde vive

`.claude/settings.json` (projeto, versionado) e `.claude/settings.local.json` (local, não versionado). Edite via `/permissions`.

```
"permissions": {
  "allow": [ "Bash(git status)", "Bash(git diff*)",
    "Bash(git push origin main)" ],
  "deny": [ "Bash(git reset --hard*)", "Bash(rm -rf*)",
    "Bash(git push --force origin main)" ]
}
```

◆ APLICAÇÃO MC

Esse é o **nosso settings.json real** (código MC-MB-063, "Workaround Classifier · permissions explícitas"). Liberamos git de leitura e push controlado; **negamos por padrão** os destrutivos: `git reset --hard`, `git filter-branch`, `rm -rf`, force-push em main. A cerca `deny` protege o substrato git (ADR-020) mesmo se algo der errado no raciocínio.

Modos de permissão (Shift+Tab alterna)

Modo	Comportamento
<code>default</code>	Pergunta na primeira vez que usa cada ferramenta
<code>plan</code>	Read-only: explora, não edita
<code>acceptEdits</code>	Aceita edições de arquivo automaticamente
<code>dontAsk</code>	Nega o que não estiver pré-aprovado
<code>bypassPermissions</code>	Pula todos os prompts — só em ambiente isolado

Sandbox — a terceira camada

Além de `allow/deny` (o que o Code tenta) e dos hooks (validação em runtime), existe a **sandbox**: restrição no nível do SO sobre quais arquivos ele lê/escreve e quais domínios acessa.

```
"sandbox": {  
  "filesystem": { "denyRead": ["~/ssh/**", "/etc/**"] },  
  "network": { "deniedDomains": ["..."] }  
}
```

▲ ATENÇÃO

Os modos permissivos (`bypassPermissions`) só em ambiente isolado/efêmero (como a web). Em TALÃO, mantenha as cercas `deny` de pé. As três camadas — permissões, hooks, sandbox — são defesa em profundidade, não redundância.

◆ APLICAÇÃO MC

A sandbox é a trava de SO ideal para a regra LGPD: negar leitura de caminhos que possam conter PII fora da stack self-hosted, e restringir domínios de rede ao confiável. Soma-se ao `mc-lint` (hook) e às cercas `deny` (permissões).

13 Sessões, checkpoints e rewind

O Code preserva o trabalho entre interações e deixa você voltar atrás com segurança.

Continuar e retomar

- `claude --continue` (ou `-c`) — retoma a sessão mais recente na pasta.
- `claude --resume` (ou `/resume`) — escolhe de uma lista de sessões anteriores.

Checkpoints e rewind

O Code cria pontos de checkpoint do estado do código ao longo da sessão, permitindo **desfazer** (rewind) para um estado anterior se uma sequência de edições não deu certo — sem precisar mexer no git manualmente.

◆ APLICAÇÃO MC

No ambiente web (onde este guia roda) o container é **efêmero**: o que não for commitado e empurrado se perde quando a sessão expira. Logo, a disciplina é **commitar e dar push** ao concluir — casando com Conventional Commits e o fluxo `main → develop → feature/MC-XXX`. Checkpoint/rewind é a rede de segurança dentro da sessão; git é a memória entre sessões.

14 Modo headless e SDK

O Code não precisa de um humano no loop. No modo **headless** (não-interativo) ele roda um prompt e devolve a resposta — pronto para pipelines, scripts e CI.

Uso básico

```
# roda um prompt e sai
claude -p "resuma as mudanças do último commit"

# saída estruturada para outro programa consumir
claude -p "liste os arquivos alterados" --output-format json

# encadeado num pipe
cat erro.log | claude -p "qual a causa raiz?"
```

SDK / Agent SDK

A mesma engine está disponível como **Claude Agent SDK** (este guia, inclusive, roda sobre ele) para construir agentes customizados em TypeScript/Python — com ferramentas, hooks e MCP programáticos.

◆ APLICAÇÃO MC

Casamento natural com o nosso **N8N self-hosted** e o backend NestJS do Igor: um nó pode chamar o Code em headless para uma tarefa determinística (ex.: classificar um documento não-sensível, gerar um checklist). **Mas:** triagem não-sensível ~70% vai no Gemini; sensível (~25%) no Llama 4 Scout self-hosted; o Claude entra na fatia ética (~5%) via MCP. Headless não muda essa divisão de águas LGPD.

15 Claude Code na web e ambientes

A versão web (claude.ai/code) roda o Code em um **container isolado e efêmero** na nuvem, não na sua máquina. É a superfície onde este próprio guia está sendo produzido.

Conceitos

Conceito	O que é
Ambiente (environment)	Config de rede, variáveis e setup do container
Política de rede	Quanto acesso externo o container tem
Sessão	Uma execução; o repo é clonado fresco no início
Gatilho (trigger)	O que iniciou: web, app, GitHub Action...
SessionStart hook	Prepara o ambiente (instala deps, testes, linters)

◆ APLICAÇÃO MC

Por ser efêmero, vale a regra de ouro: **nada importante sobrevive sem commit + push**. Um **SessionStart hook** (skill session-start-hook) pode garantir que, ao abrir uma sessão web, o ambiente MC já suba pronto — dependências, lint, mc-lint armado. Documentação oficial: code.claude.com/docs/en/claude-code-on-the-web.

▲ ATENÇÃO

A política de rede do ambiente governa o que sai do container. Em sessão que toque qualquer coisa próxima de dado sensível, isso importa — confirme a política antes.

16 Integração com GitHub

O Code se integra ao GitHub para revisar PRs, abrir PRs, comentar e reagir a CI — via GitHub Actions e via o servidor MCP do GitHub.

O que dá para fazer

- **Revisão de PR** com `/review` ou via Action que comenta automaticamente.
- **Monitorar um PR**: assinar eventos (comentários, CI, reviews) e responder/consertar — incluindo "deixar o CI verde".
- **Criar PR** — só quando explicitamente pedido.

◆ APLICAÇÃO MC

Útil no backend do Igor (NestJS/TS/MySQL): o Code pode revisar um PR contra as convenções MC — TypeScript strict, sem `any` implícito, DTOs obrigatórios, **dado sensível nunca em log**, Conventional Commits. Combinado com o hook mc-lint, vira uma malha dupla: lint determinístico + revisão de PR.

▲ ATENÇÃO

Conteúdo de comentário/PR/CI é **dado externo** — pode tentar redirecionar o agente. Tratar com a mesma desconfiança do ADR-016 (Anti-Injection / Radar Galileu). Ser frugal: só comentar quando necessário.

17 IDE, atalhos e modo Vim

O Code vive dentro do seu editor e responde a teclado.

Extensões de IDE

VS Code e JetBrains: o Code enxerga o arquivo aberto, a seleção e os diagnósticos do editor; mostra diffs inline. Para o backend NestJS do Igor, é o habitat natural.

Atalhos essenciais (terminal)

Atalho	Ação
Esc	Interrompe o agente
Esc Esc	Edita a mensagem anterior / volta atrás
Ctrl+C	Cancela
Shift+Tab	Alterna modos (inclui Plan Mode)
↑ / ↓	Navega o histórico de prompts
# no início	Adiciona à memória (CLAUDE.md)
! no início	Modo bash direto

Modo Vim

`/vim` liga edição estilo Vim no campo de prompt (modos normal/insert, navegação por palavras). As teclas são customizáveis em `~/.claude/keybindings.json`.

18 Ferramentas nativas

Por baixo dos comandos, o Code age através de um conjunto de ferramentas embutidas. Saber quais existem ajuda a entender (e a permissionar) o que ele faz.

Ferramenta	O que faz
Read	Lê arquivos (texto, imagem, PDF, notebooks)
Edit	Edição cirúrgica de trecho de arquivo
Write	Cria/sobrescreve arquivo
Glob	Encontra arquivos por padrão (<code>**/*.ts</code>)
Grep	Busca conteúdo por regex (ripgrep)
Bash	Roda comandos de shell (foreground/background)
WebSearch	Busca na web
WebFetch	Lê uma URL específica
Task / Agent	Dispara um subagente
TodoWrite	Mantém a lista de tarefas da sessão

◆ APLICAÇÃO MC

Proof-First na prática usa essas ferramentas: `Grep` / `Read` para citar **arquivo:linha**, `WebSearch` / `WebFetch` para fonte verificável, `Bash` para gerar o **SHA-256** de uma fonte. A skill `/mc-dossie-proof-first` orquestra exatamente isso. `Read` de PDF serve para ler peças e normas; **nunca** para ingerir PII de cidadão fora da stack self-hosted.

19 Output styles, status line e /fast

Output styles

Ajustam o **estilo de comportamento/resposta** do Code para diferentes propósitos (ex.: mais explicativo para ensino, mais conciso para execução). Configuráveis e customizáveis.

Status line

A linha de status na base do terminal é customizável (skill statusline-setup) — pode mostrar branch, modelo, contexto usado, ou um indicador próprio.

/fast

Liga o **modo rápido**: Opus com saída acelerada (não rebaixa para um modelo menor). Disponível no Opus 4.6/4.7/4.8. Bom para iteração veloz sem perder qualidade.

◆ APLICAÇÃO MC

Uma status line MC poderia exibir o **Context Budget** em % (gatilho visual para `/compact` em 60%) e a branch `feature/MC-XXX` ativa. `/fast` acelera trabalho operacional R1; em cunhagem R2/R3, profundidade > velocidade.

20 /loop e recorrência

O recurso da semana: `/loop` roda um prompt ou comando de barra **repetidamente, em intervalos**.

```
/loop <intervalo> <prompt-ou-comando>
# intervalo opcional (padrão 10m)

/loop 5m /watchdog
/loop 10m verifique movimentações e me avise se algo mudou
```

Serve para **monitoramento e polling** — acompanhar um deploy, uma fila, um status. Não use para tarefa pontual.

◆ APLICAÇÃO MC

Encaixe direto na **fase E5 (Watchdog)** da jornada: `/loop` reexecutando `/watchdog` para revarrer casos aguardando análise INSS em intervalo regular.

× LINHA VERMELHA

Monitorar movimentação processual é dever (CDC art. 6º III — comunicar sempre). Mas **jamais** deixar o loop "prometer prazo de concessão": análise INSS ~52,67d (LAI SEI 24851867), variável. O loop reporta fato, não promessa.

A Apêndice — Mapa do Canivete

Recurso nativo → aplicação direta no Meu Cumpadre. A lâmina certa para cada tarefa.

Recurso	Aplicação MC	Já existe?
CLAUDE.md	Doutrina viva: D>C>V, Proibições, precificação	Sim · v3.0
Skills	/novo-cliente, /watchdog, /squad-r2, /squad-r3, /mc-pre-selagem...	Sim · família
Subagente	sister-ancia (revisão adversarial 9 eixos)	Sim
Hook (Stop)	mc-lint block-mode (Proibições) + mc-audit	Sim
Permissões	Cercas deny: rm -rf, reset --hard, force-push main	Sim · MB-063
MCP	GitHub, Supabase, n8n, Adobe	Sim
Plan Mode	/orquestracao · pré-refactor de ADR · Spec-Driven	Parcial
/loop	Watchdog E5 recorrente	Disponível
Headless/SDK	Nós N8N · tarefas determinísticas não-sensíveis	A explorar
GitHub Actions	Revisão de PR do backend NestJS	A explorar
SessionStart hook	Subir ambiente web MC pronto	A explorar
Status line	Context Budget % + branch	A explorar

B Apêndice — Régua MC ao operar o Code

Sete travas que valem para **qualquer** recurso deste guia. A ferramenta amplia o operador; nunca suspende a regra.

1. **D > C > V** — dignidade do hipervulnerável acima de tudo.
2. **Firewall OAB** — MC é atividade-meio. Nunca exercer advocacia, prometer resultado, fazer match comercial. Atividade-fim = Dra. Juliana Pereira de Melo.
3. **Nunca prometer prazo de concessão INSS** — "48h" = sprint interno E0→E4 com doc completa; análise INSS é variável.
4. **Proof-First** — zero dado fabricado; fonte verificável (Lei + Evidência + SHA-256) ou **[FONTE PENDENTE]**. Cross-referenciar sempre.
5. **LGPD** — CPF/NB/CID/CNIS só na stack self-hosted; senha gov.br só no Bitwarden Org MC. Nunca por MCP genérico, nunca em log, nunca no ClickUp.
6. **Gate de Vault** — Code escreve no Cérebro 1 livre; no Vault (Cérebro 2) só com "aprovado para vault". OAB/LGPD/PII ⇒ Rito R3 (Juliana).
7. **Cercas deny de pé** — destrutivos bloqueados; commit+push para sobreviver ao container efêmero.

◆ SÍNTESE

O Claude Code é o canivete. A doutrina MC é a mão que o segura. **Lucro é combustível. Causa é destino. Jogo é infinito.**

MC-GUIA-ClaudeCode-Canivete-v1_0-2026-0605 · Documento de capacitação · Zona Verde (método/ferramenta, sem PII) · Gate R1 · Fonte de verdade sobre recursos: documentação oficial code.claude.com/docs. Recursos e comandos podem variar com a versão do Claude Code — confirme com /help.